

DAY 2 · MODULE 6 · 35 MIN

Authoring Custom Function Tools in MAF

Craft, testing, and patterns

What you're doing now

- Module 4 covered WHY function calling works
- Module 5 covered NOT writing tools (Toolbox)
- Module 6 is where you WRITE tools well

This module + the lab: from 'I can pass a function to tools=[...]' to 'I can author, test, and ship a production function tool.'

Focus: Python (labs are Python-only). C# equivalents referenced from Module 4.

The four ways to author a function tool

Pattern	When to reach for it
Bare function	Simplest case — no decorator needed
@tool decorator	Explicit name, description, approval_mode
@tool(schema=...)	Full Pydantic or JSON schema — enums, constraints, complex validation
Class-based tools	Multiple tools that share state (client handles, config, cached data)

Bare functions are fine for most cases. Reach for @tool when you need control, classes when you need shared state.

Pattern 1 — Bare function (the default)

Fastest path. Any Python function becomes a tool.

```
from typing import Annotated
from pydantic import Field

def get_weather(
    location: Annotated[str, Field(description="The location to get the weather for.")],
) -> str:
    """Get the weather for a given location."""
    return f"The weather in {location} is cloudy with a high of 15°C."

agent = Agent(client=..., instructions="...", tools=[get_weather])
```

- Docstring → description; Field(description=...) → parameter description; type hints → schema
- Use for: simple tools with straightforward params, no runtime context, no approval gate

Pattern 2 — @tool decorator (when you need control)

```
from agent_framework import tool

@tool(
    name="weather_tool",
    description="Retrieves weather information for any location",
    approval_mode="never_require",
)
def get_weather(
    location: Annotated[str, Field(description="The location to get the weather for.")],
) -> str:
    return f"The weather in {location} is cloudy with a high of 15°C."
```

When to use:

- Tool name differs from the Python function name
- Description belongs in the decorator (not a docstring)
- Explicit approval_mode (never_require / always_require)
- You'll add schema=... later

Pattern 3 — Explicit schemas

For complex inputs, define the schema as a Pydantic model:

```
class TicketInput(BaseModel):
    title: Annotated[str, Field(description="Short ticket title")]
    body: Annotated[str, Field(description="Full description of the problem")]
    priority: Annotated[
        Literal["low", "med", "high"],
        Field(description="Ticket priority"),
    ] = "med"

@tool(
    name="create_ticket",
    description="Create a support ticket. Use when the user reports a problem needing a human engineer.",
    schema=TicketInput,
    approval_mode="always_require",
)
def create_ticket(title: str, body: str, priority: str = "med") -> str:
    ticket_id = ticket_system.create(title, body, priority)
    return f"Created ticket {ticket_id}"
```

Literal['low', 'med', 'high'] becomes a schema enum. Model can't pass invalid priorities.

Pattern 4 — Class-based tools (shared state)

When several tools share a client, cache, or configuration:

```
class TicketTools:
    def __init__(self, api_client: TicketAPIClient) -> None:
        self._client = api_client

    def create_ticket(
        self,
        title: Annotated[str, "Short ticket title"],
        body: Annotated[str, "Full description"],
    ) -> str:
        """Create a support ticket."""
        return self._client.create(title, body)

    def lookup_status(self, ticket_id: Annotated[str, "The ticket ID"]) -> str:
        """Look up the status of a ticket by ID."""
        return self._client.get_status(ticket_id)

tools_instance = TicketTools(api_client=my_client)
agent = Agent(client=..., tools=[tools_instance.create_ticket, tools_instance.lookup_status])
```

Class attributes (self._client) are hidden from the model. Bound methods are what the agent sees.

Async is a first-class citizen

Function tools can be async def:

```
@tool
async def lookup_status(ticket_id: str) -> str:
    """Look up the status of a ticket by ID."""
    async with aiohttp.ClientSession() as session:
        async with session.get(f"{API_BASE}/tickets/{ticket_id}") as r:
            data = await r.json()
    return f"Ticket {ticket_id}: {data['status']}"
```

- MAF awaits your async tool — same schema surface, same invocation flow
- Rule of thumb: if your tool does I/O (HTTP, DB, disk), make it async

Runtime context (recap from Module 4)

When you need per-call runtime values, add `ctx: FunctionInvocationContext`:

```
@tool(approval_mode="never_require")
def get_user_orders(
    limit: Annotated[int, Field(description="Max number of orders")],
    ctx: FunctionInvocationContext,
) -> str:
    """Get the current user's recent orders."""
    user_id = ctx.kwargs["user_id"] # supplied by caller, not model
    orders = orders_service.list(user_id, limit=limit)
    return json.dumps(orders)

await agent.run(
    "What are my recent orders?",
    function_invocation_kwargs={"user_id": "user_123"},
)
```

ctx is hidden from the model. Use for tenancy, session, logging correlation IDs.

Return values — what the model sees

Return types matter more than you might realize:

Return type	What the model sees	When to use
str	Raw string	Simple text results, error messages
dict / list	JSON-serialized	Structured data the model reasons over
Pydantic model	JSON via <code>.model_dump_json()</code>	Typed, validated returns
Custom class	Serialized via MAF default	Only if you must

Rule: if the tool returns structured data the model will act on, prefer Pydantic. If it's just an outcome, str is fine.

Bad return shape = model can't parse the result = wrong follow-up action.

Error contracts (recap from Module 4)

```
# 1. Return an error string – model can incorporate
@tool
def lookup_status(ticket_id: str) -> str:
    try:
        return ticket_system.get_status(ticket_id)
    except NotFound:
        return f"Ticket {ticket_id} not found. Verify the ID and try again."

# 2. Return a structured error – model can route or retry
@tool
def lookup_status(ticket_id: str) -> dict:
    try:
        return {"status": ticket_system.get_status(ticket_id)}
    except NotFound:
        return {"error": "not_found", "message": "Verify the ticket ID"}

# 3. Raise an exception – MAF surfaces the exception message
@tool
def lookup_status(ticket_id: str) -> str:
    return ticket_system.get_status(ticket_id)    # raises if not found
```

Pick per tool. Prefer 'error as data' when you want the model to recover.

Docstring as prompt — the four-part template

You saw this in Module 4. Here's the discipline for authoring:

```
@tool
def create_ticket(title: str, body: str, priority: str) -> str:
    """Create a support ticket for a problem that needs a human engineer.

    Use this when the user reports a problem you cannot answer from
    documentation. Do NOT use for general questions.

    Priority must be one of: low, med, high. Default med.
    Returns the created ticket ID.
    """
```

- What — one line describing the action
- When to call it
- When NOT to call it — the disambiguation
- What comes back — return shape

Testing tools in isolation

Your tool is just a Python function. Test it like one.

```
# tests/test_create_ticket.py
from labs.day2.tools import create_ticket

def test_create_ticket_returns_id():
    result = create_ticket(title="Login fails", body="500 on POST /login", priority="high")
    assert result.startswith("Created ticket ")

def test_create_ticket_invalid_priority_raises():
    with pytest.raises(ValidationError):
        create_ticket(title="X", body="Y", priority="urgent")
```

- Test the tool WITHOUT an agent — faster feedback, no LLM calls
- Catches schema and logic bugs before they get to the model
- Pydantic validates args BEFORE your function body runs — the second test catches this automatically

Golden-set testing — the tool level

You learned eval Day 2 Module 3 for retrieval. Same shape for tool use:

```
# tools_golden_set.jsonl
{"query": "My login is failing with a 500", "expected_tool": "create_ticket",
 "expected_args": {"priority": "high"}}
{"query": "What are your business hours?", "expected_tool": null}
{"query": "Look up ticket 12345", "expected_tool": "lookup_status",
 "expected_args": {"ticket_id": "12345"}}
```

- Run the agent against each query. Verify: which tool (or none), what args
- Catches regressions when you change a description or add a competing tool

Day 4's evaluation anchor extends this pattern to full workflows.

Composition — using one agent as another's tool

MAF supports wrapping an agent as a tool. Useful for domain sub-agents:

```
# Sub-agent: focused on weather
weather_agent = Agent(client=..., instructions="You answer weather questions.", tools=[get_weather])

# Main agent uses the weather sub-agent as a tool
main_agent = Agent(
    client=...,
    instructions="You answer questions in French. Use the weather agent for weather questions.",
    tools=[weather_agent.as_tool(name="ask_weather", description="Ask the weather sub-agent")],
)
```

The sub-agent's name and description become the tool signature. This is a preview of Day 4's multi-agent patterns.

Common traps (deeper than Module 4)

- Docstring style + description in decorator — mismatch. One or the other, not both differing.
- Optional params without defaults — model doesn't know what to pass; add a default.
- Return None — no signal for the model. Return "" or "OK" explicitly.
- Blocking I/O in a sync tool — blocks the agent. Use async def for I/O.
- Skipping validation — Pydantic schema is your first line of defense against invalid args.
- Assuming type hints are enough — write Field(description=...) for parameters; the model reads it.

What you'll build in Part B of the lab

Your Day 2 lab has three parts. Part B is a Module 6 hands-on:

- Two mock function tools: `create_ticket` and `lookup_status`
- Test each in isolation before wiring to an agent
- Wire them into the Day 1 docs-assistant agent
- Add a small tool-use eval (which tool did the model pick?)
- Iterate on descriptions until the model picks correctly

Day 3 swaps the mock ticket tool for a real Azure DevOps MCP server — same conceptual pattern, real backend.

Takeaways

- Four authoring patterns: bare function, `@tool`, `@tool(schema=...)`, class-based. Pick the smallest that fits.
- Async is first-class. If your tool does I/O, make it `async def`.
- Docstring is a prompt. Four-part template: what, when, when-NOT, return shape.
- Test tools in isolation with `pytest`. Add a tool-use golden set for the agent-level check.
- Return shape matters — prefer `Pydantic` for structured returns.
- Errors as data when the model should recover.

Next: Day 2 lab kickoff — the docs-assistant with ticket triage + evaluation.